



Scotty App

Proyecto **Intermodular**



Manuela · José Ramón · Iván · José Luís

IES Mutxamel · Curso 2025/2026 · 1º DAW

ABSTRACT (José Luis)

This project developed Scotty App, a web based educational platform designed to help students learn programming through short interactive challenges, progression levels, points, and digital rewards. The platform addressed the common problem of low motivation and poor engagement in traditional, theory-heavy programming lessons by offering a more practical and game-like learning experience. The project was developed using an iterative and incremental methodology supported by Scrum, allowing the team to organize work in stages and validate each part progressively. The main technologies used included HTML5, CSS3, JavaScript, Java, and SQL Server, together with design and coordination tools such as Figma, GitHub, and Trello. The final result was a functional MVP that included user registration and login, a basic challenge system, experience and level progression, and a progress dashboard, all presented through a responsive interface adapted to different devices.

Índice Contenido:

1. Identificación del Proyecto	6
1.1. Título del Proyecto:	6
1.2. Datos del Grupo e Integrantes:.....	6
1.3. Roles Iniciales del Equipo:.....	6
2. Introducción y Justificación.....	7
2.1. Objetivo del Proyecto (Enfoque SMART)	7
2.2. Justificación del Proyecto (Problema y relevancia)	8
2.3. Análisis de lo Existente (Benchmarking de 3 apps).....	8
3. Requisitos Funcionales	9
3.1. Descripción del Problema a Resolver	9
3.2. Funcionalidades Principales (Alcance MVP)	9
3.3. Tipo de Usuarios y Acciones (Perfiles)	11
4. Análisis y Diseño Lógico.....	11
4.1. Diagrama de la Arquitectura de la Aplicación.....	11
4.1.1. Diagrama Simple de Arquitectura (Cajitas)	13
4.2. Diagrama de Casos de Uso (UML).	14
4.3. Diagrama de Clases (UML)	16
4.4. Modelo de Datos.....	18
4.4.1. Diagrama Entidad-Relación (E-R).....	19
4.4.2. Modelo Relacional	20
4.4.3. Script SQL de creación de tablas	20
4.5. Diseño de Interfaz y Prototipo Funcional (Landing Page)	21
4.5.1. Estructura Semántica HTML5	23
4.5.2. Estilos y Diseño Responsive (Tailwind CSS)	23
4.5.3. Formulario de Registro y Validaciones	24
4.5.4. URL de Despliegue Público (Hosting CDMON)	24
5. Metodología de Desarrollo	25

5.1. Descripción del Ciclo de Vida del Software	25
5.2. Metodología Ágil Elegida (Scrum)	25
5.4. Coordinación y Seguimiento del Proyecto	27
6. Codificación.....	28
6.1. Entorno de programación.....	28
6.2. Lenguajes y herramientas.....	29
6.3. Aspectos relevantes de la implementación	30
7. Plan de Negocio	32
7.1. Público Objetivo.....	32
7.2. Utilidad Social o Empresarial — (José Luis).....	33
7.3. Posibles Costes y Beneficios.....	33
8. Manual de Usuario	33
8.1. Descripción general de la aplicación.....	33
8.2. Acceso al sistema	34
8.3. Registro e inicio de sesión	34
8.4. Navegación por la aplicación.....	34
8.5. Uso de las funcionalidades principales.....	35
9. Requisitos e Instalación.....	35
9.1. Requisitos del sistema.....	35
9.2. Instalación y ejecución	36
10. Conclusiones	37
10.1. Conclusiones sobre el trabajo realizado	37
10.2. Posibles ampliaciones y mejoras	37
11. Anexos: Diseño Preliminar	38
11.1. Descripción Paso a Paso del Caso de Uso Principal.....	38
11.2. Mockups de 2–3 Pantallas Clave	39
11.3. Prevención de Riesgos Laborales (PRL)	39
11.3.1. Identificación de riesgos asociados al desarrollo (Teletrabajo).	40
11.3.2. Medidas preventivas aplicables al trabajo con equipos informáticos	40
11.3. 3. Facilitación de material y responsabilidades	40

12. Seguimiento del Proyecto (enlaces externos de gestión)	40
12.1. URL del Tablero de Trello (Gestión de tareas)	41
12.2. URL del Repositorio de GitHub	41
12.2.1. Gestión de Ramas y Commits.....	43
12.3. Control de Versiones y README.md	43
1. Bibliografía y webgrafía.....	44

Índice Figuras:

Figura 1 - Diagrama de arquitectura de la aplicación

Figura 2 - Diagrama de casos de uso

Figura 3 - Diagrama de clases

Figura 4 – Diagrama Entidad-Relación

Figura 5 - Diseño de la Landing Page

Figura 6 – Diagrama Simple de Arquitectura

Figura 7 - Mockup de pantallas clave

Figura 8 - Captura del tablero Trello

Figura 9 - Captura del repositorio GitHub

1. Identificación del Proyecto

1.1. Título del Proyecto:

Scotty App: Plataforma web ramificada para el aprendizaje de lenguajes de programación.

1.2. Datos del Grupo e Integrantes:

El equipo se identifica como Scotty Team y se estructura bajo una metodología de trabajo cooperativo:

- José Luis: Analista de Negocio e Investigación.
- José Ramón: Analista de Sistemas y Requisitos.
- Iván: UX/UI Designer y Diseño de Interacción.
- Manuela: DevOps, Infraestructura, Seguridad y Scrum Master.

1.3. Roles Iniciales del Equipo:

Para este hito, se han asignado responsabilidades alineadas con las etapas de desarrollo:

José Luis — Investigación y Viabilidad (Analista de Negocio)

- Responsable de la fase de análisis de mercado y la viabilidad del proyecto.
- Justificación: Explica el problema real que resuelve Scotty App y su relevancia educativa.
- Benchmarking: Realiza el estudio comparativo de aplicaciones similares e identifica las ventajas de nuestra propuesta.
- Plan de Negocio: Define el público objetivo, la utilidad social y el análisis de costes y beneficios.

José Ramón — Requisitos y Estructura (Analista de Sistemas)

- Define qué hace la aplicación y cómo se organizan internamente los datos.
- Requisitos: Lista las funciones del MVP y describe los problemas a resolver.
- Perfiles: Identifica los tipos de usuario y sus permisos de acceso.
- Modelado: Diseña el diagrama de clases UML, el modelo entidad-relación, el modelo relacional y el script SQL de creación de tablas.

Iván — Diseño y Experiencia de Usuario (UX/UI Designer)

- Encargado de la parte visual y de la interacción entre el usuario y el sistema.
- Interfaces: Crea los mockups de las pantallas clave en Figma.
- Landing Page: Desarrolla la página funcional en HTML y Tailwind CSS y la despliega en hosting CDMON.
- Flujo: Describe detalladamente el caso de uso principal paso a paso.

Manuela — Infraestructura, Seguridad y Metodología (DevOps / Scrum Master)

- Asegura que las herramientas técnicas y el entorno de trabajo sean estables y seguros.
- Arquitectura: Diseña el diagrama de arquitectura de la aplicación y el diagrama de casos de uso UML.
- Metodología: Define el ciclo de vida del software, aplica el marco de trabajo Scrum y gestiona el tablero de Trello y el backlog de tareas.
- Salud Laboral: Redacta el plan de Prevención de Riesgos Laborales aplicado al desarrollo de software.
- Consolidación: Realiza la revisión final del documento para asegurar la integridad de la entrega.

2. Introducción y Justificación

2.1. Objetivo del Proyecto (Enfoque SMART)

El objetivo principal de Scotty App es diseñar y desarrollar una plataforma web gamificada que permita a estudiantes de ESO, Bachillerato y Formación Profesional aprender lenguajes de programación mediante retos interactivos con sistema de puntos, niveles y ranking.

- Específico (S): Construir un MVP funcional de Scotty App que incluya: un sistema de registro e inicio de sesión, un motor de retos interactivos con al menos 5 lenguajes de programación (HTML, CSS, JavaScript, Java y SQL), un sistema de experiencia (XP) con niveles progresivos, un panel de progreso del alumno y un ranking global de usuarios.
- Medible (M): El éxito del proyecto se medirá verificando que el MVP cumpla los siguientes criterios concretos: mínimo 5 retos funcionales implementados (uno por lenguaje), persistencia de datos de usuario y progreso en SQL Server, formulario de registro operativo con al menos 3 validaciones, landing page desplegada y accesible desde URL pública, y documentación técnica completa con todos los diagramas exigidos (casos de uso, clases, E-R, relacional y script SQL).
- Alcanzable (A): El proyecto es viable porque el equipo cuenta con los conocimientos adquiridos durante el primer curso de DAW: programación en Java, bases de datos con SQL Server, maquetación web con HTML/CSS/JavaScript, y gestión de proyectos con Scrum. Además, el alcance del MVP se ha limitado deliberadamente a funcionalidades que podemos implementar dentro de las aproximadamente 32 horas de desarrollo disponibles por persona.
- Relevante (R): La propuesta responde a un problema real que hemos experimentado como estudiantes: aprender a programar con métodos puramente teóricos resulta desmotivador. Scotty App convierte ese aprendizaje en algo práctico e inmediato, con feedback visual y recompensas que incentivan la constancia. Además, la gamificación educativa es una tendencia creciente en el sector EdTech.
- Temporal (T): El desarrollo del MVP se organiza en los siguientes plazos: Hito I (idea y grupo) completado en diciembre 2025, Hito II (análisis y planificación) completado en febrero 2026, Hito III (diseño técnico) con entrega el 1 de abril de 2026, Hito IV (codificación e implementación) del 1 al 23 de abril de 2026, y entrega final con presentación el 10 de mayo de 2026.

2.2. Justificación del Proyecto (Problema y relevancia)

Cualquiera que haya intentado aprender a programar desde cero sabe lo que es enfrentarse a una pantalla en blanco sin saber por dónde empezar. Las clases teóricas explican conceptos, pero muchas veces falta esa parte práctica inmediata que te haga ver que lo que estás aprendiendo sirve para algo.

Nosotros mismos, como estudiantes de primer curso de DAW, hemos notado que los compañeros que más se enganchan a la programación son los que encuentran una forma de practicar que no se sienta como "deberes". Cuando resuelves un reto y ves el resultado al instante, la motivación cambia completamente.

Scotty App nace de esa experiencia. Queremos ofrecer una herramienta donde aprender a programar se sienta más como jugar que como estudiar, pero sin perder el rigor del contenido. La plataforma presenta ejercicios reales de programación envueltos en mecánicas de juego: ganas puntos, subes de nivel, compites en un ranking y desbloqueas recompensas. No se trata de trivializar el aprendizaje, sino de hacerlo más atractivo y sostenible en el tiempo.

Desde el punto de vista educativo, Scotty App también ofrece valor para los profesores: pueden crear retos adaptados a su temario, agrupar alumnos en equipos y hacer seguimiento del progreso de la clase. Esto convierte la plataforma en un complemento real para el aula, no solo en una app más de autoaprendizaje.

2.3. Análisis de lo Existente (Benchmarking de 3 apps)

Antes de definir Scotty App, analizamos tres plataformas que combinan educación y gamificación para entender qué funciona, qué falta y dónde podemos aportar algo diferente.

- Kahoot! es probablemente la herramienta de gamificación más conocida en las aulas. Funciona como un sistema de quizzes en tiempo real donde los alumnos compiten respondiendo preguntas con puntuación y ranking en vivo. Es extremadamente motivadora y fácil de usar, y muchos profesores ya la utilizan. Sin embargo, Kahoot! no permite programar código real: se limita a preguntas tipo test. Para aprender conceptos generales funciona bien, pero no sirve para practicar la escritura de código, que es la competencia fundamental que queremos trabajar.
- CodeCombat es una plataforma que sí permite programar código real dentro de un videojuego RPG. El alumno escribe instrucciones en Python, JavaScript u otros lenguajes para controlar a su personaje y superar niveles. La gamificación es profunda: tiene XP, gemas, equipamiento y una historia. El problema es que gran parte del contenido avanzado es de pago, y la plataforma está pensada para el autoaprendizaje individual, no para integrarse con la dinámica de un aula ni adaptarse al temario de un profesor concreto.
- SoloLearn ofrece cursos interactivos de muchos lenguajes de programación con un editor integrado en el navegador. Tiene una comunidad global enorme y un sistema de XP y certificados. Su debilidad principal es que no está pensada para centros educativos: no permite al profesor crear contenido personalizado, agrupar alumnos ni hacer seguimiento del progreso de una clase.

Síntesis y posicionamiento de Scotty App:

Criterio	Kahoot!	CodeComba	SoloLearn	ScottyApp
Código Real	No	Si	Si	Si (editor/bloque)
Gamificación	Alta (quiz)	Alta (RPG)	Media (XP)	Alta (XP, niveles, tokens, ranking)
Gratuito	Parcial	Parcial	Parcial	100% gratuito (MVP)
Personalizable por profesor	Sí (quizzes)	No	No	Sí (retos, equipos, seguimiento)
Integración con el aula	Alta	Baja	Baja	Alta (profesor + equipos)
Seguimiento del progreso	Básico	Individual	Individual	Panel profesor + alumno

Lo que diferencia a Scotty App es la combinación de tres cosas que ninguna de las tres plataformas analizadas ofrece juntas: programación real con gamificación profunda, personalización por parte del profesor y seguimiento del progreso a nivel de aula. No pretendemos competir con plataformas con años de desarrollo, pero sí cubrir un hueco concreto: una herramienta gratuita y adaptable que un profesor de programación pueda usar directamente en su clase.

3. Requisitos Funcionales

3.1. Descripción del Problema a Resolver

El principal problema que detectamos es que aprender a programar desde cero es muy árido. La teoría suelta aburre y muchos alumnos lo dejan porque no ven una aplicación práctica rápida. Queremos solucionar esto con una web donde la lógica de programación se aprenda jugando, con retos cortos que te den una recompensa visual inmediata y te motiven a seguir subiendo de nivel.

3.2. Funcionalidades Principales (Alcance MVP)

A continuación se formalizan los requisitos funcionales de Scotty App, organizados por perfil de usuario. Cada requisito sigue la nomenclatura RF (Requisito Funcional) con un código único, y está redactado en infinitivo para garantizar claridad y verificabilidad. Esta estructura se basa en la metodología aplicada en la práctica de la taquilla inteligente del módulo de Entornos de Desarrollo.

Requisitos funcionales del Invitado:

El Invitado es cualquier persona que accede a la plataforma sin haberse registrado. Su interacción es limitada y temporal: puede explorar el contenido y probar retos, pero no se guarda ningún dato al cerrar la sesión.

- RF-I01: El sistema permitirá explorar el catálogo de juegos y niveles disponibles sin necesidad de registro.
- RF-I02: El sistema permitirá probar retos en modo temporal, sin guardado de progreso ni puntuación.
- RF-I03: El sistema permitirá acceder al formulario de registro para crear una cuenta de alumno.
- RF-I04: El sistema mostrará un mensaje informativo indicando que el progreso no se guarda sin registro.

Requisitos funcionales del Alumno:

El Alumno es un usuario registrado que hereda todas las acciones del Invitado. Además, tiene acceso a funcionalidades que requieren autenticación: guardado de progreso, acumulación de XP y tokens, consulta de ranking y canje de recompensas.

- RF-A01: El sistema permitirá registrarse proporcionando nombre, correo electrónico y contraseña.
- RF-A02: El sistema permitirá iniciar sesión mediante correo electrónico y contraseña.
- RF-A03: El sistema permitirá cerrar sesión de forma segura.
- RF-A04: El sistema permitirá editar los datos del perfil de usuario (nombre, contraseña, avatar).
- RF-A05: El sistema permitirá jugar retos interactivos con guardado automático de progreso.
- RF-A06: El sistema acumulará puntos de experiencia (XP) por cada reto completado correctamente.
- RF-A07: El sistema calculará y mostrará el nivel del alumno en función de la XP acumulada.
- RF-A08: El sistema permitirá consultar el panel de progreso personal (niveles completados, XP, racha).
- RF-A09: El sistema permitirá consultar el ranking global de usuarios ordenado por XP.
- RF-A10: El sistema acumulará tokens canjeables como recompensa por retos superados.
- RF-A11: El sistema permitirá canjear tokens por recompensas disponibles en la tienda virtual.

Requisitos funcionales del Profesor:

El Profesor es una especialización del Alumno. Además de las funcionalidades heredadas, puede crear retos, gestionar equipos de alumnos y consultar el progreso de su clase.

- RF-P01: El sistema permitirá al profesor crear retos personalizados asociados a un nivel y lenguaje.
- RF-P02: El sistema permitirá al profesor agrupar alumnos en equipos de trabajo.
- RF-P03: El sistema permitirá al profesor asignar retos específicos a un equipo de alumnos.
- RF-P04: El sistema permitirá al profesor consultar un panel de seguimiento con el progreso de sus alumnos.
- RF-P05: El sistema permitirá al profesor editar o eliminar los retos que haya creado.

Requisitos funcionales del Administrador:

El Administrador es un actor independiente responsable de la gestión global del sistema: contenido educativo, usuarios, ranking y recompensas.

- RF-D01: El sistema permitirá al administrador crear, editar y eliminar juegos y niveles del catálogo.
- RF-D02: El sistema permitirá al administrador gestionar las cuentas de usuario (activar, desactivar, eliminar).
- RF-D03: El sistema permitirá al administrador moderar el ranking global (eliminar puntuaciones fraudulentas).
- RF-D04: El sistema permitirá al administrador validar o rechazar solicitudes de canje de tokens.
- RF-D05: El sistema permitirá al administrador gestionar el catálogo de recompensas de la tienda virtual.
- RF-D06: El sistema permitirá al administrador consultar estadísticas globales de uso de la plataforma.

3.3. Tipo de Usuarios y Acciones (Perfiles)

- Invitado: Cualquier persona que accede sin registrarse. Puede explorar el catálogo de juegos y probar los retos, pero su sesión es completamente temporal: si cierra el navegador, todo desaparece. No se guarda ni puntuación ni progreso.
- Alumno: Usuario registrado. Hereda las acciones del Invitado y además puede iniciar sesión, editar su perfil, jugar con guardado de progreso, acumular tokens y XP, consultar su posición en el ranking global y canjear recompensas.
- Profesor: Especialización del Alumno. Además de todas las acciones del alumno, puede crear retos basados en el material de clase, agrupar a sus alumnos en equipos, asignarles retos específicos y consultar un panel de seguimiento con el progreso de su clase.
- Administrador: Gestiona el sistema de forma global. Se encarga del mantenimiento del contenido educativo (crear, editar y eliminar juegos y niveles), la administración de usuarios, la moderación del ranking y la validación de solicitudes de canje cuando es necesario.

4. Análisis y Diseño Lógico

4.1. Diagrama de la Arquitectura de la Aplicación.

Descripción de los componentes:

Frontend (Interfaz de Usuario):

- Representa la interacción con el alumno.
- Tecnologías: HTML, CSS y JavaScript.
- Funcionalidades:
 - ×1 Mostrar mapa de niveles y retos.
 - ×2 Editor de ejercicios (simulación, no ejecución real de código).
 - ×3 Perfil de usuario, ranking y tienda de recompensas.

- Idea futura empresa: se puede evolucionar a React o Angular, con diseño responsive y animaciones gamificadas.

Backend (Servidor de Orquestación):

- Responsable de recibir y gestionar las peticiones del frontend.
- Tecnologías posibles para el proyecto DAW: Java.
- Funcionalidades:
 - ×1 Gestión de usuarios y login.
 - ×2 Guardado de progreso de alumnos.
 - ×3 Devolver ejercicios y niveles al frontend.
- Idea futura empresa: podría convertirse en API REST profesional, con autenticación segura, validación de datos y soporte multiusuario escalable.

Base de Datos (Almacenamiento persistente):

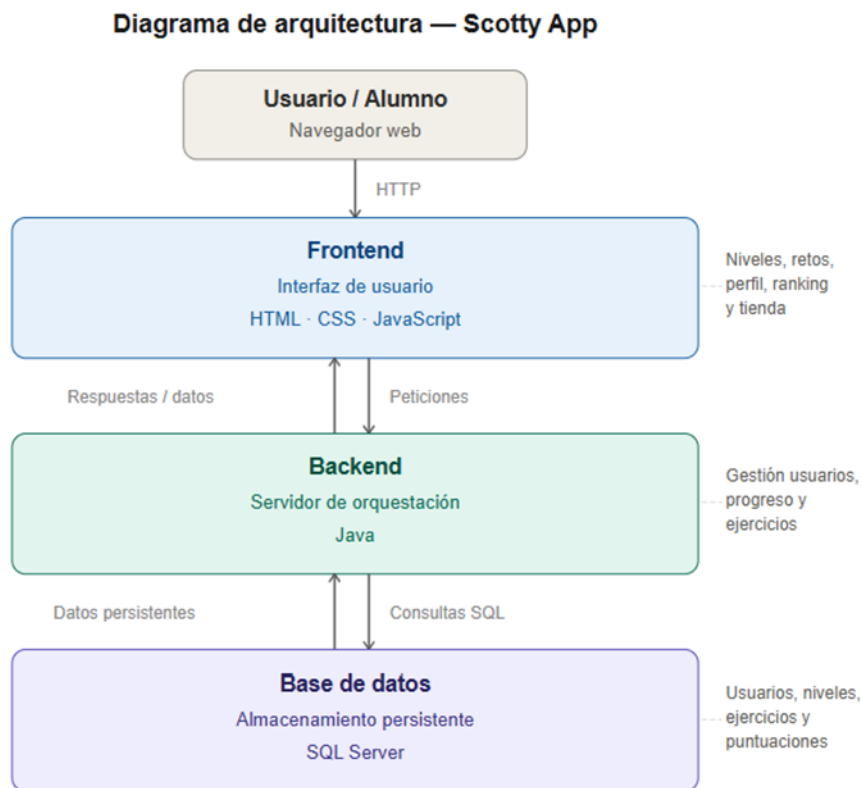
- Almacena toda la información de manera persistente:
 - ×1 Usuarios
 - ×2 Niveles y ejercicios
 - ×3 Progreso y puntuaciones
 - ×4 Recompensas
- SQLServer.
- Idea futura empresa: escalar a MySQL, PostgreSQL o incluso un servicio en la nube, con soporte para múltiples usuarios y consultas complejas.

Flujo de funcionamiento del **sistema**:

- El alumno accede a la plataforma desde el navegador.
- El frontend muestra los ejercicios y la interfaz de usuario.
- El alumno envía respuestas o selecciona soluciones.
- El backend recibe las peticiones y registra los datos en la base de datos.
- El frontend muestra resultados, niveles desbloqueados y recompensas.

Bloque y flujo visual			
Bloque	Función	Qué hace	Tecnología
Frontend	Interfaz de usuario	Mostrar niveles, retos, perfil, ranking, tienda	HTML, CSS, JavaScript
Backend	Servidor/logica	Gestión de usuarios, niveles, ejercicios y puntuación	Java
Base de datos	Almacen de información	Guardar usuarios, niveles, ejercicios y progresos	SQL Server

Figura 1 - Diagrama de arquitectura de la aplicación



4.1.1. Diagrama Simple de Arquitectura (Cajitas)

El apartado 4.1 describe la arquitectura de la aplicación en detalle técnico. Este anexo lo complementa con una representación visual simplificada (tipo "cajitas") que resume los tres componentes principales del sistema y el flujo de datos entre ellos. El objetivo es ofrecer una vista de alto nivel que permita entender la estructura de Scotty App de un vistazo.

El diagrama se estructura en tres capas:

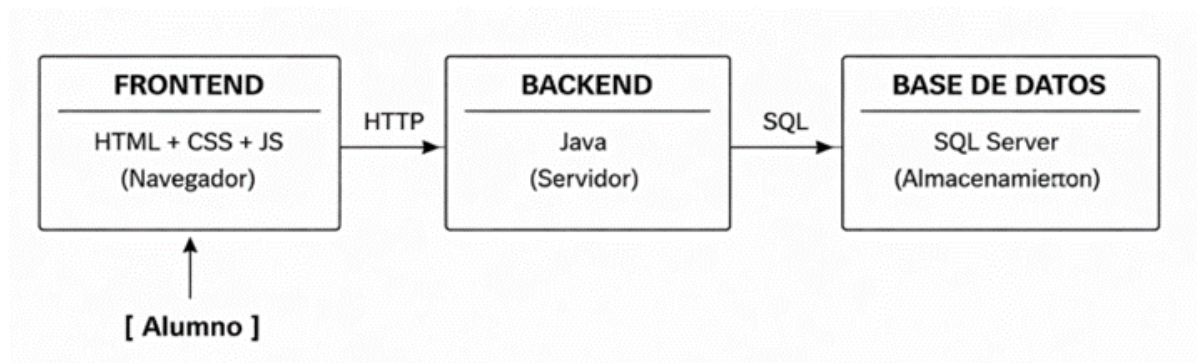
Capa 1 — Frontend (Navegador del usuario): Es el punto de entrada del alumno. Se ejecuta en el navegador y muestra la interfaz gráfica: mapa de niveles, editor de retos, perfil, ranking y tienda de recompensas. Tecnologías: HTML5, CSS3 (Tailwind CSS) y JavaScript.

Capa 2 — Backend (Servidor de aplicación): Recibe las peticiones del frontend y ejecuta la lógica de negocio: autenticación de usuarios, validación de respuestas, cálculo de XP y gestión de niveles. Tecnología: Java.

Capa 3 — Base de Datos (Almacenamiento persistente): Almacena de forma permanente todos los datos del sistema: usuarios, lenguajes, cursos, quizzes, preguntas, respuestas, resultados y progreso. Tecnología: Microsoft SQL Server.

Flujo de datos: El alumno interactúa con el Frontend, que envía peticiones HTTP al Backend. El Backend procesa la lógica y consulta o actualiza la Base de Datos. La respuesta recorre el camino inverso: Base de Datos → Backend → Frontend → Pantalla del alumno.

Figura 6 – Diagrama Simple de Arquitectura.



Este esquema sigue el modelo de arquitectura en tres capas (presentación, lógica de negocio y persistencia), estudiado en el módulo de Sistemas Informáticos. Al ser un modelo iterativo e incremental, este diagrama puede evolucionar en futuras fases del proyecto.

4.2. Diagrama de Casos de Uso (UML).

Antes de ponernos a programar cualquier cosa, necesitamos tener claro qué va a hacer nuestra aplicación y quién va a interactuar con ella. Para eso usamos el diagrama de casos de uso: nos permite representar visualmente qué acciones puede realizar cada tipo de usuario dentro del sistema, sin entrar todavía en cómo se va a implementar. Como dice el temario, un caso de uso describe cómo un actor interactúa con el sistema para lograr un objetivo concreto.

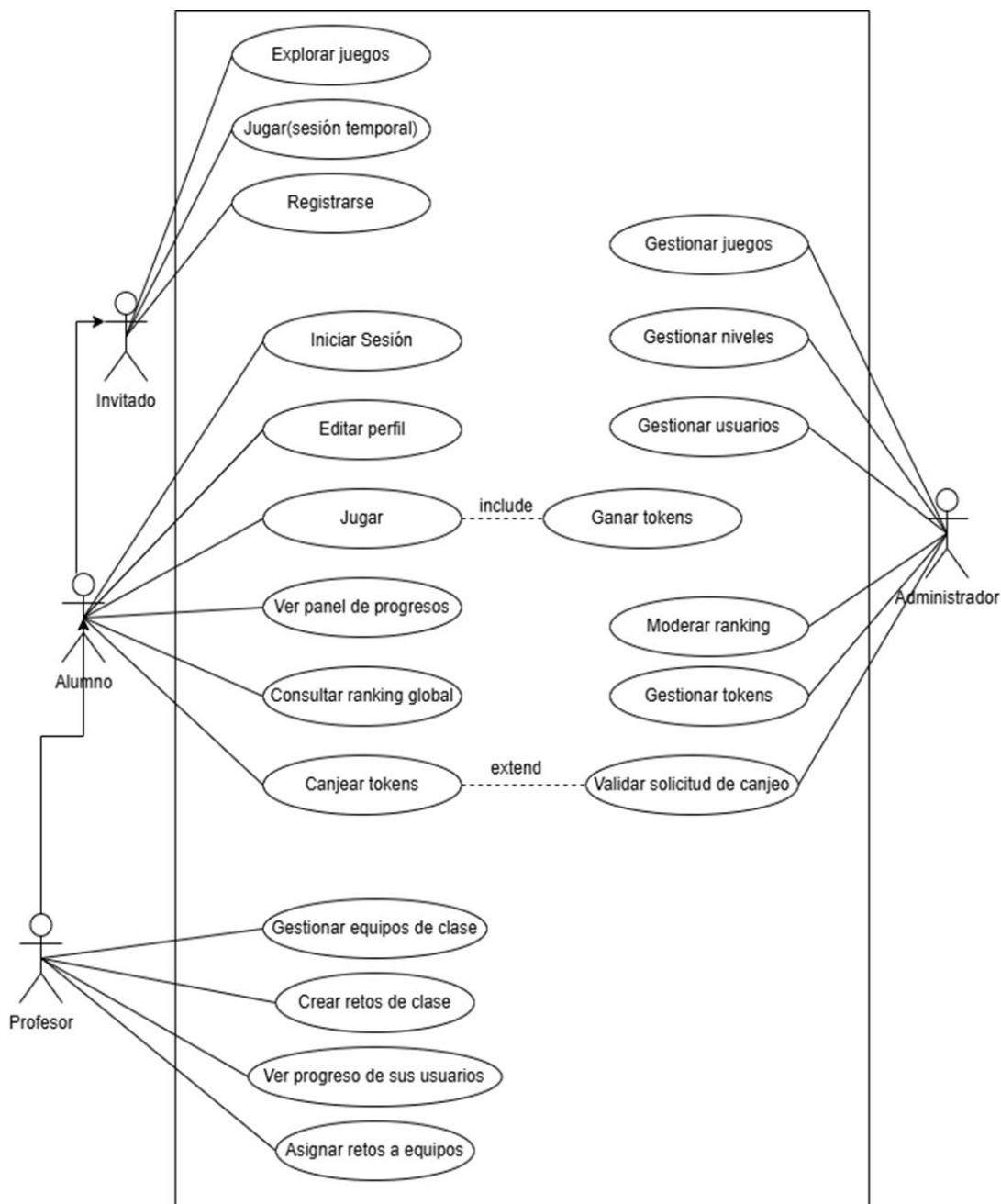
Actores identificados

Hemos identificado cuatro actores. El criterio ha sido pensar en quién usa realmente la plataforma y con qué nivel de acceso.

- El Invitado es cualquier persona que llega a Scotty App sin registrarse. Puede explorar el catálogo y probar los juegos, pero su sesión es completamente temporal: si cierra el navegador, todo desaparece. No se guarda ni puntuación ni progreso. Es el gancho para que alguien quiera registrarse.
- El Alumno es una especialización del Invitado, lo que en UML se representa con una relación de generalización. Hereda todo lo que puede hacer el Invitado y además tiene acceso a las funcionalidades que requieren cuenta: iniciar sesión, editar su perfil, jugar con guardado de progreso, acumular tokens, consultar su posición en el ranking global y canjear recompensas.
- El Profesor es una especialización del Alumno, y por tanto hereda también todo lo que hereda el Alumno del Invitado. Además de jugar y ver su propio progreso, puede crear retos basados en el material que trabaja en clase, agrupar a sus alumnos en equipos, asignarles esos retos y ver un panel de seguimiento con el progreso de su clase. Es el actor que conecta Scotty App con el entorno educativo real.

- El Administrador es un actor independiente que gestiona el sistema de forma global. Se encarga del mantenimiento del contenido educativo (juegos y niveles), la gestión de usuarios, la moderación del ranking y la validación de solicitudes de canje cuando es necesario.

Figura 2 - Diagrama de casos de uso



Relaciones del diagrama

Hemos aplicado los tres tipos de relaciones que explica la UP02.

La generalización aparece dos veces: del Alumno hacia el Invitado, y del Profesor hacia el Alumno. Funciona igual que la herencia en los diagramas de clases: cada actor más específico tiene todas las acciones del más general más las propias.

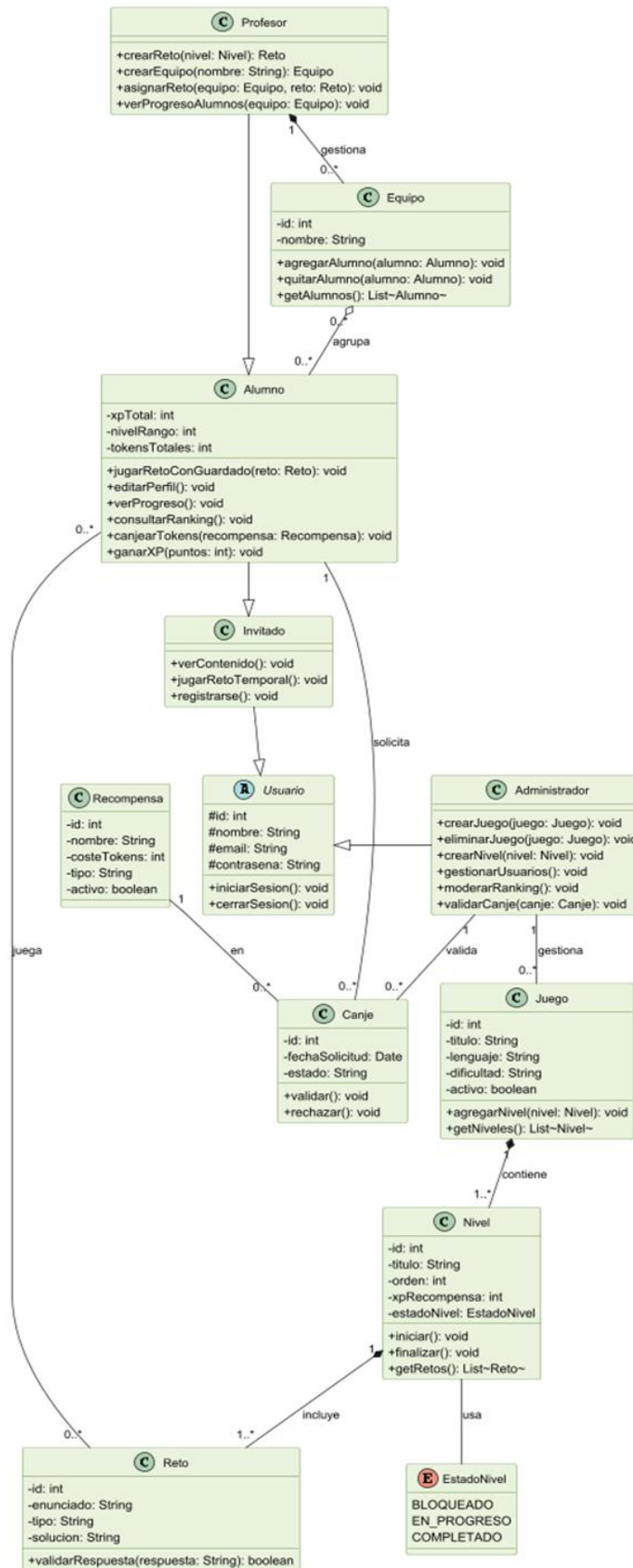
La relación «include» se usa donde la ejecución de un caso de uso siempre implica otro obligatoriamente. Jugar un reto con guardado siempre requiere haber iniciado sesión, y siempre otorga XP al completarse. Sin estas dos condiciones la lógica del juego no funciona.

La relación «extend» conecta "Canjear tokens" con "Validar solicitud de canje". La intervención del Administrador en un canje no ocurre siempre, depende del tipo de recompensa, así que se modela como extensión opcional.

4.3. Diagrama de Clases (UML)

El diagrama de clases representa la estructura estática del sistema, mostrando las clases principales, sus atributos, métodos y las relaciones entre ellas. Se ha generado con PlantUML utilizando la directiva `skinparam classAttributeIconSize 0` para que las visibilidades (+, -, #) se interpreten correctamente como texto y no como iconos.

Figura 3 - Diagrama de clases



A continuación se describe cada clase y su función dentro del sistema:

Usuario (clase abstracta): Clase base que contiene los atributos comunes a todos los usuarios del sistema (id, nombre, email, contraseña). Define los métodos `iniciarSesion()` y `cerrarSesion()`.

Invitado: Hereda de Usuario. Representa a los usuarios no registrados. Puede ver contenido, jugar retos en modo temporal y registrarse.

Alumno: Hereda de Invitado. Añade atributos de progreso (`xpTotal`, `nivelRango`, `tokensTotales`) y métodos para jugar con guardado, editar perfil, consultar ranking y canjear recompensas.

Profesor: Hereda de Alumno. Añade la capacidad de crear retos, gestionar equipos y consultar el progreso de los alumnos asignados.

Administrador: Clase independiente que gestiona el sistema: crear y eliminar juegos, gestionar usuarios, moderar el ranking y validar canjes.

Juego: Representa una unidad temática de aprendizaje (por ejemplo, "Introducción a HTML"). Contiene una lista de niveles.

Nivel: Cada juego se compone de niveles ordenados. Cada nivel otorga una recompensa en XP y tiene un estado (BLOQUEADO, EN_PROGRESO o COMPLETADO definido por la enumeración `EstadoNivel`). Contiene una lista de retos.

Reto: Ejercicio concreto con enunciado, tipo (selección, código, bloques) y solución. Dispone del método `validarRespuesta()` que devuelve un booleano.

Recompensa: Objeto canjeable por tokens (avatares, insignias, etc.). Tiene un coste en tokens y un estado activo/inactivo.

Canje: Registro de cada solicitud de canje realizada por un alumno. Puede ser validado o rechazado por el Administrador.

Equipo: Agrupación de alumnos creada por un profesor para asignar retos colectivos.

Las relaciones principales son: la jerarquía de herencia sigue el patrón Invitado → Alumno → Profesor. Un Juego contiene de 1 a muchos Niveles (composición). Un Nivel incluye de 0 a muchos Retos. Un Alumno puede realizar de 0 a muchos Canjes, y cada Canje puede ser validado por el Administrador. Un Profesor gestiona de 0 a muchos Equipos, y cada Equipo agrupa de 0 a muchos Alumnos.

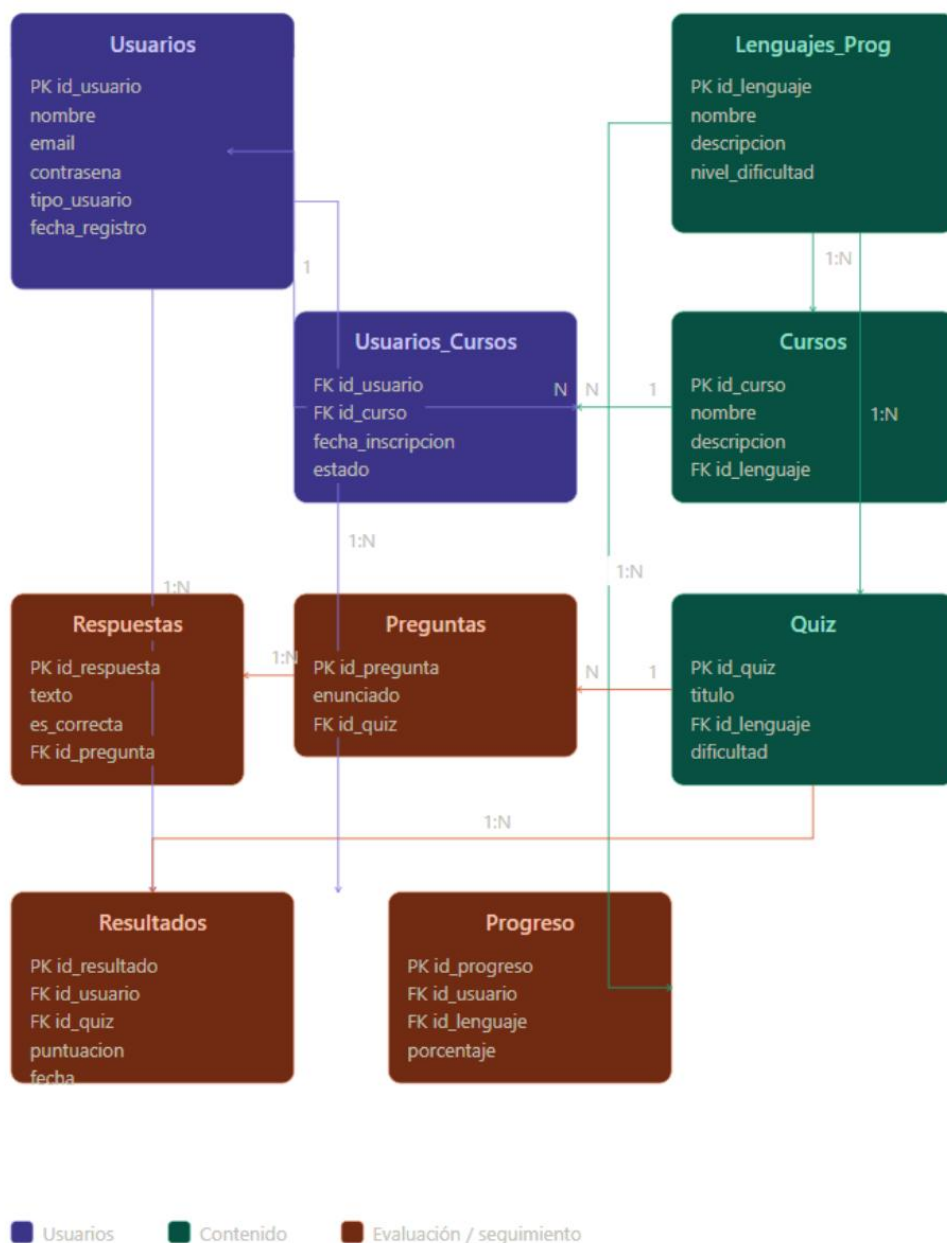
4.4. Modelo de Datos

El modelo de datos de Scotty App ha sido diseñado para dar soporte a todas las funcionalidades definidas en los requisitos funcionales. La base de datos se implementa en Microsoft SQL Server, utilizando T-SQL como lenguaje de definición y manipulación de datos. El modelo está compuesto por 9 tablas organizadas en tres bloques lógicos: gestión de usuarios, contenido educativo y evaluación/seguimiento del progreso.

4.4.1. Diagrama Entidad-Relación (E-R)

El diagrama E-R representa gráficamente las entidades del sistema, sus atributos y las relaciones entre ellas. Se ha elaborado siguiendo la notación estudiada en el módulo de Bases de Datos (RA2: ER Model and Design).

Figura 4 – Diagrama Entidad-Relación



Las entidades principales son: Usuarios, Lenguajes_Programacion, Cursos, Quiz, Preguntas, Respuestas, Resultados y Progreso. La tabla intermedia Usuarios_Cursos resuelve la relación muchos a muchos entre usuarios y cursos.

Las relaciones identificadas son:

- Un usuario puede inscribirse en muchos cursos y un curso puede tener muchos usuarios (N:M, resuelta por Usuarios_Cursos).
- Un lenguaje de programación puede tener muchos cursos, quizzes y registros de progreso asociados (1:N).
- Un quiz contiene muchas preguntas, y cada pregunta tiene muchas respuestas (1:N en cadena).
- Un usuario puede tener muchos resultados de quiz y muchos registros de progreso (1:N).

4.4.2. Modelo Relacional

A partir del diagrama E-R se ha derivado el modelo relacional. Cada entidad se transforma en una tabla con sus atributos, claves primarias (PK) y claves foráneas (FK):

Tabla	Atributos	Claves foráneas
Usuarios	id_usuario (PK), nombre, email, contraseña, tipo_usuario, fecha_registro	—
Lenguajes_Programacion	id_lenguaje (PK), nombre, descripcion, nivel_dificultad	—
Cursos	id_curso (PK), nombre, descripcion, id_lenguaje (FK)	id_lenguaje → Lenguajes_Programacion
Usuarios_Cursos	id_usuario (PK,FK), id_curso (PK,FK), fecha_inscripcion, estado	id_usuario → Usuarios, id_curso → Cursos
Quiz	id_quiz (PK), titulo, id_lenguaje (FK), dificultad	id_lenguaje → Lenguajes_Programacion
Preguntas	id_pregunta (PK), enunciado, id_quiz (FK)	id_quiz → Quiz
Respuestas	id_respuesta (PK), texto, es_correcta, id_pregunta (FK)	id_pregunta → Preguntas
Resultados	id_resultado (PK), id_usuario (FK), id_quiz (FK), puntuacion, fecha	id_usuario → Usuarios, id_quiz → Quiz
Progreso	id_progreso (PK), id_usuario (FK), id_lenguaje (FK), porcentaje_completado	id_usuario → Usuarios, id_lenguaje → Lenguajes_Programacion

4.4.3. Script SQL de creación de tablas

El script SQL implementa el modelo relacional en Microsoft SQL Server. Las tablas están ordenadas respetando las dependencias de claves foráneas, de forma que el script puede ejecutarse directamente sin errores de orden.

El script completo se encuentra en [GitHub](#)

Aspectos técnicos destacables: se utiliza VARCHAR(MAX) para campos de texto largo como descripciones y enunciados, NVARCHAR(15) para campos que pueden contener caracteres especiales como el nivel de dificultad, BIT para el campo booleano es_correcta de Respuestas, clave primaria compuesta (id_usuario, id_curso) en Usuarios_Cursos para resolver la relación N:M, y DECIMAL(5,2) en porcentaje_completado de Progreso.

4.5. Diseño de Interfaz y Prototipo Funcional (Landing Page)

Como parte del diseño del sistema, el equipo ha desarrollado una Landing Page funcional que sirve como primera versión real de la interfaz de Scotty App. No se trata de un mockup ni de un diseño estático, sino de una página web completamente operativa construida con HTML, CSS y JavaScript.

La Landing Page cumple con todos los requisitos mínimos establecidos para esta entrega:

Menú de navegación funcional: La página incluye una barra de navegación fija en la parte superior (sticky) con cuatro enlaces internos (Inicio, ¿Qué es?, Mini juegos y Registro) y un botón de llamada a la acción "Empezar gratis". En dispositivos móviles, el menú se transforma en un icono hamburguesa que despliega un menú vertical controlado por JavaScript. Al hacer scroll, el menú móvil se cierra automáticamente para no entorpecer la navegación.

Estructura HTML5 semántica: El documento utiliza correctamente las etiquetas semánticas de HTML5: <header> para la barra de navegación, <nav> para los enlaces, <section> para cada bloque de contenido (hero, características, mini juegos y registro), <form> para el formulario de alta y <footer> para el pie de página. Cada sección está identificada con un id único que permite la navegación por anclas.

Tailwind CSS como framework de estilos: Se ha utilizado Tailwind CSS cargado por CDN, complementado con una hoja de estilos personalizada (styles.css) que define componentes reutilizables como los botones, las tarjetas de quiz, el formulario y las animaciones. La elección de Tailwind se debe a que es una tecnología actual muy utilizada en entornos profesionales, tal como recomienda el enunciado de la entrega.

Diseño responsive: La página se adapta a tres rangos de pantalla. En móvil el contenido se apila en una sola columna y aparece el menú hamburguesa. En tablet las tarjetas se reorganizan en dos columnas y los botones del hero pasan a horizontal. En escritorio se muestra el menú completo y las tarjetas se distribuyen en tres columnas.

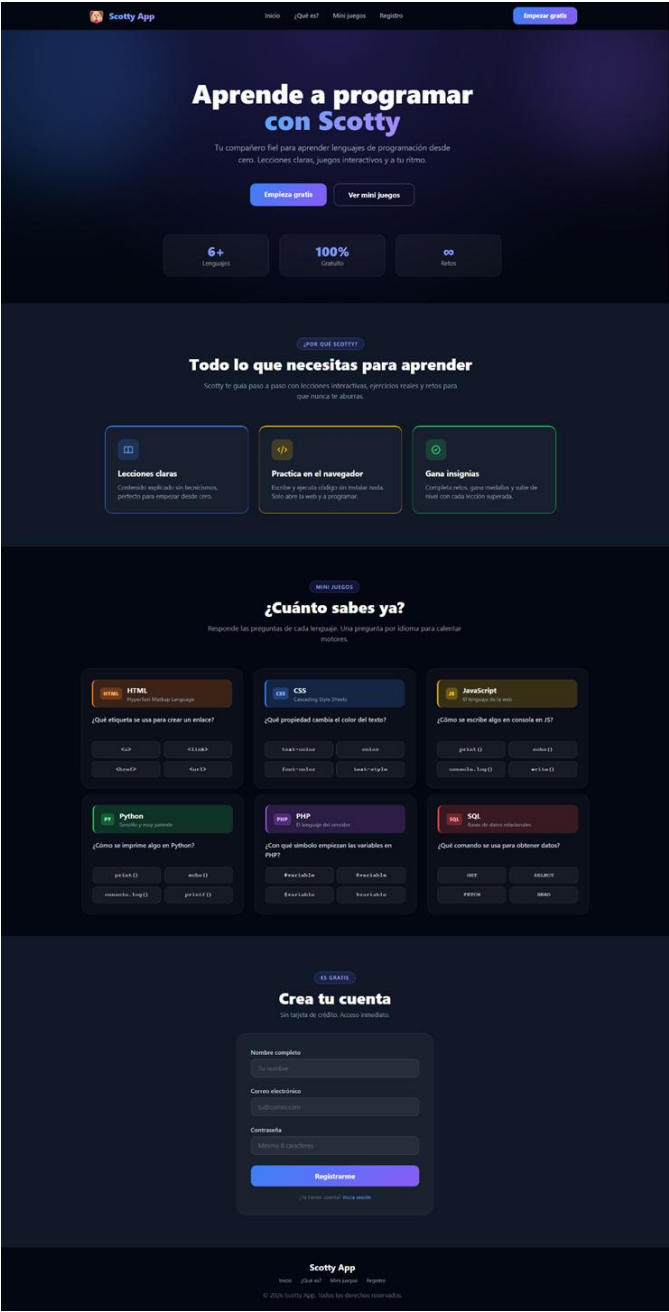
Formulario de registro: La sección de registro recoge nombre completo, correo electrónico y contraseña. Incluye validación en el lado del cliente mediante JavaScript: el nombre debe tener al menos 3 caracteres, el email debe cumplir un formato válido (verificado con expresión regular) y la contraseña debe tener un mínimo de 8 caracteres. Los errores se muestran debajo de cada campo con un estilo visual claro (borde rojo), y cuando el campo es correcto se marca en verde. Al superar todas las validaciones aparece un mensaje de éxito.

Mini juegos interactivos: Además del formulario, la Landing Page incluye una sección de mini juegos con seis preguntas tipo quiz (una por lenguaje: HTML, CSS, JavaScript, Python, PHP y SQL). Cada pregunta tiene cuatro opciones y el sistema da feedback visual inmediato: verde para la respuesta correcta, rojo para la

incorrecta. Al completar las seis preguntas se muestra una puntuación final con un mensaje personalizado según el número de aciertos. Esta sección sirve como demostración funcional del concepto de Scotty App.

Código estructurado y organizado: El proyecto está separado en tres archivos: index.html (estructura y contenido), styles.css (estilos personalizados) y app.js (lógica del menú, quiz y validaciones). Cada archivo está comentado con secciones claramente delimitadas.

Figura 5 - Diseño de la Landing Page



4.5.1. Estructura Semántica HTML5

La Landing Page de Scotty App se ha desarrollado siguiendo las buenas prácticas de estructura semántica HTML5 estudiadas en el módulo de Lenguaje de Marcas (RA2 — HTML y CSS).

El documento utiliza las siguientes etiquetas semánticas:

<header>: Contiene la barra de navegación principal con el logotipo de Scotty App, los enlaces de navegación (Inicio, ¿Qué es?, Mini juegos, Registro) y el botón "Empezar gratis". La barra es sticky (permanece fija al hacer scroll) y utiliza un efecto de desenfoque para mantener la legibilidad.

<nav>: Elemento semántico dentro del header que agrupa los enlaces de navegación. Incluye una versión de escritorio (lista horizontal que se muestra a partir de 768px) y una versión móvil (menú hamburguesa desplegable controlado por JavaScript).

<section>: Se utilizan cuatro secciones principales, cada una identificada con un id único para la navegación interna por anclas: #inicio (hero con presentación y estadísticas), #que-es (características de la plataforma), #juegos (mini juegos interactivos de quiz) y #registro (formulario de alta).

<form>: Formulario de registro con atributo novalidate para gestionar la validación mediante JavaScript personalizado. Contiene tres campos de entrada (nombre, email, contraseña) con sus respectivos <label> asociados mediante el atributo for/id.

<footer>: Pie de página con la marca de Scotty App, enlaces de navegación secundarios y aviso de copyright.

El documento comienza con <!DOCTYPE html> y el atributo lang="es", indicando que el contenido está en español. <meta charset="UTF-8"> garantiza la correcta visualización de caracteres especiales, y <meta name="viewport"> asegura la adaptación a dispositivos móviles.

4.5.2. Estilos y Diseño Responsive (Tailwind CSS)

Para el diseño visual se ha utilizado Tailwind CSS cargado mediante CDN, complementado con una hoja de estilos personalizada (styles.css) para componentes específicos del proyecto.

Paleta de colores: Se ha elegido una estética dark mode con fondo oscuro (bg-gray-950 / bg-gray-900) y texto blanco/gris claro. Los acentos de color usan un gradiente azul-morado (de #60a5fa a #a78bfa) que se aplica al logotipo, botones principales y textos destacados.

Sistema responsive: Tailwind utiliza un sistema mobile-first. La Landing Page se adapta así: en móvil (< 768px) el menú se oculta y aparece el icono hamburguesa, las tarjetas de quiz se apilan en una columna y los botones del hero se apilan verticalmente. En tablet (≥ 640px) las tarjetas se reorganizan en 2 columnas y los botones pasan a horizontal. En escritorio (≥ 768px) se muestra el menú completo y las tarjetas de features y quiz se muestran en 3 columnas.

Componentes CSS personalizados: Se han creado clases propias en styles.css: .btn-primary y .btn-secondary (botones con efecto hover de elevación y sombra), .quiz-card (tarjetas de juego con borde sutil y hover), .feature-card (tarjetas de características con borde superior de color), .form-card, .form-input, .form-label y .form-error (elementos del formulario), y .navbar-blur (efecto de desenfoque de la navbar).

Animaciones: Se han implementado tres animaciones CSS: `.logo-bounce` (rebote continuo en el icono), `.float-anim` (flotación suave para elementos decorativos) y `.aparecer` (fade-in con desplazamiento vertical para los mensajes de feedback del quiz y el resultado final).

4.5.3. Formulario de Registro y Validaciones

El formulario de registro recoge tres datos: nombre completo, correo electrónico y contraseña. La validación se realiza en el lado del cliente mediante JavaScript (`app.js`), desactivando la validación nativa del navegador con `novalidate`.

Validación del nombre: Se comprueba que tenga al menos 3 caracteres (tras eliminar espacios con `trim()`). Si no cumple, se muestra "El nombre debe tener al menos 3 caracteres" y se aplica la clase `.error` al campo (borde rojo). Si es válido, se aplica `.valido` (borde verde).

Validación del email: Se usa la expresión regular `/^[^\s@]+@[^\s@]+\.[^\s@]+$/` para verificar el formato básico de correo electrónico. Si no es válido, se muestra "Introduce un correo electrónico válido".

Validación de la contraseña: Se exige una longitud mínima de 8 caracteres. Si no se cumple, se muestra "La contraseña debe tener al menos 8 caracteres".

Comportamiento en caso de éxito: Si los tres campos son válidos, se muestra un mensaje de confirmación ("Registro completado correctamente. Bienvenido/a a Scotty App.") con fondo verde, se resetea el formulario y se eliminan las clases de estilo. Se usa `e.preventDefault()` para evitar el envío real, ya que en esta fase no hay backend que procese los datos.

4.5.4. URL de Despliegue Público (Hosting CDMON)

Respecto al despliegue público en CDMON, no ha sido posible completar la subida de la Landing Page al hosting en el plazo de esta entrega debido a problemas técnicos en la configuración del servicio.

No obstante, la Landing Page está completamente desarrollada y funcional en local, y todo el código fuente está disponible en el repositorio de GitHub de la organización Scotty-App. Cualquier persona con acceso al repositorio puede descargar los tres archivos (`index.html`, `styles.css` y `app.js`), abrirlos en un navegador y comprobar que la página funciona correctamente con todas las funcionalidades descritas en los apartados anteriores.

El equipo se compromete a completar el despliegue en CDMON durante el inicio del Hito IV, de forma que la URL pública esté operativa antes de la siguiente entrega. Tailwind CSS se carga mediante CDN, por lo que no requiere instalación adicional en el servidor: basta con subir los tres archivos al hosting para que la página funcione.

5. Metodología de Desarrollo

5.1. Descripción del Ciclo de Vida del Software

El ciclo de vida del software es el proceso que sigue un proyecto desde que surge la idea hasta que el producto está terminado y en funcionamiento. En la asignatura de Entornos de Desarrollo hemos estudiado los distintos modelos que existen para organizar este proceso.

Para ScottyApp hemos identificado las siguientes fases, que además están directamente relacionadas con los hitos de entrega definidos en el proyecto intermodular:

- **Análisis:** Definimos qué tiene que hacer la plataforma, quiénes son los usuarios y qué problemas resuelve. Esta fase dio lugar a los requisitos funcionales del proyecto, recogidos en el apartado de Requisitos Funcionales de esta documentación.
- **Diseño:** Planificamos la arquitectura de la aplicación, el modelo de datos y el prototipo visual. Esta fase incluye la elaboración de los diagramas entidad-relación y modelo relacional (Bases de Datos), la definición de la arquitectura y los sistemas necesarios para el despliegue (Sistemas Informáticos) y la elaboración del plan de negocio de la plataforma (IPE). La prevención de riesgos laborales asociados al desarrollo, trabajada en FOL, se abordó durante el Hito 2 como parte del análisis y planificación del proyecto. En Entornos de Desarrollo aprendimos que un buen diseño previo evita errores costosos durante el desarrollo.
- **Implementación:** Cada miembro del equipo desarrolla su parte según la distribución de tareas acordada. Esta fase incluye la maquetación con HTML y CSS (Lenguaje de Marcas), el desarrollo de la lógica de programación (Programación) y la creación real de la base de datos mediante el script SQL de creación de tablas en SQL Server (Bases de Datos).
- **Pruebas:** Verificamos que la aplicación funciona correctamente. En Entornos de Desarrollo trabajamos técnicas de prueba como caja negra y análisis de valores límite, que aplicamos para validar los formularios y la lógica de la plataforma. Esta fase incluye también la revisión y mejora del código desarrollado en Java durante el curso.
- **Despliegue:** Publicamos la aplicación en un servidor real. Iván se encargó del hosting en CDMON, poniendo en práctica conceptos de despliegue vistos en clase.
- **Mantenimiento:** Aunque en este proyecto académico es limitado, contemplamos la posibilidad de corregir errores y añadir mejoras tras la entrega.

5.2. Metodología Ágil Elegida (Scrum)

Para gestionar el desarrollo de ScottyApp hemos elegido Scrum, una metodología ágil estudiada en la asignatura de Entornos de Desarrollo dentro del bloque de metodologías de desarrollo de software.

Scrum se basa en dividir el trabajo en periodos cortos con objetivos concretos, al final de los cuales se obtiene un avance verificable del producto. Nosotros hemos adaptado esta metodología usando el término Hitos en lugar de Sprints, manteniendo la misma filosofía de trabajo incremental.

Planificación de Hitos (Sprints):

Hito	Contenido	Fechas (aproximadas)
Hito I	Composición del grupo e idea del proyecto	04/12/2025 – 22/12/2025
Hito II	Análisis y planificación del proyecto	08/01/2026 – 15/02/2026
Hito III	Diseño técnico: arquitectura, modelo de datos e interfaz	19/02/2026 – 01/04/2026
Hito IV	Codificación, manual de usuario, instalación y conclusiones	01/04/2026 – 23/04/2026
Entrega final	Presentación del proyecto completo	10/05/2026

Roles Scrum:

Product Owner: José Luis. Responsable de definir las prioridades del proyecto y asegurar que el producto responde a las necesidades del usuario. Lleva el plan de negocio y la justificación del proyecto.

Scrum Master: Manuela. Responsable de que el equipo siga la metodología, de facilitar las reuniones y de hacer seguimiento del estado de las tareas.

Equipo de Desarrollo: Iván y José Ramón. Responsables de la implementación técnica: maquetación, base de datos, control de versiones y despliegue.

Ceremonias Scrum aplicadas:

Sprint Planning (Planificación del Hito): Al inicio de cada hito el equipo se reúne para revisar las tareas pendientes del backlog, decidir cuáles entran en ese hito y asignar responsables. Esto queda reflejado en el tablero Kanban de Trello. Los hitos fueron establecidos por la profesora como marco general del proyecto, y dentro de cada hito somos nosotros quienes organizamos la división de tareas y el ritmo de trabajo.

Weekly Scrum: El equipo se reúne semanalmente para que cada miembro comente en qué está trabajando, si tiene algún bloqueo y qué hará antes de la siguiente reunión. Estas reuniones nos permiten detectar problemas a tiempo y redistribuir carga si algún miembro necesita apoyo.

Sprint Review: Al finalizar cada hito el equipo revisa lo que se ha completado, comprueba que cumple los requisitos definidos y actualiza el tablero de tareas.

Sprint Retrospective: Al cerrar cada hito reflexionamos brevemente sobre qué ha funcionado bien, qué ha fallado y qué podemos mejorar en el siguiente. Un ejemplo concreto fue la reorganización del equipo a mitad del proyecto, que nos obligó a redistribuir tareas y ajustar la planificación.

5.3. Planificación Inicial de Tareas y Roles

Desde el inicio del proyecto dividimos el trabajo teniendo en cuenta las habilidades de cada miembro y buscando una carga equitativa. La asignación quedó recogida en el índice del proyecto, donde cada apartado tiene un responsable.

Los roles y responsabilidades asumidos por cada miembro son:

- **Iván** se encargó de todo lo relacionado con la parte visual y de despliegue: la estructura semántica en HTML5, los estilos y diseño responsive con CSS, el formulario de registro con sus validaciones y el hosting público en CDMON. También lidera la identificación del proyecto y los mockups de pantallas clave.
- **José Ramón** asumió la parte técnica y de datos: los requisitos funcionales, el diagrama de clases UML, el modelo de datos completo con el diagrama entidad-relación, el modelo relacional y el script SQL de creación de tablas.
- **José Luis** se centró en la justificación del proyecto, el análisis de aplicaciones similares (benchmarking) y el plan de negocio completo.
- **Manuela** trabajó el análisis y diseño lógico (diagrama de arquitectura de la aplicación y diagrama de casos de uso UML), la prevención de riesgos laborales y la bibliografía. Tras la reorganización del equipo asumió también toda la metodología de desarrollo.

El control de versiones en GitHub fue una responsabilidad rotativa entre todos los miembros del equipo, de forma que todos participaron en el mantenimiento del historial del proyecto.

5.4. Coordinación y Seguimiento del Proyecto

Para coordinar el trabajo del equipo utilizamos dos herramientas principales:

Kanban con Trello como complemento visual a Scrum:

Usamos Trello como tablero Kanban para visualizar el estado de las tareas en todo momento. Es importante aclarar que Kanban no sustituye a Scrum en nuestro proyecto, sino que funciona como herramienta de apoyo visual para gestionar el flujo de trabajo dentro de cada hito.

El tablero está estructurado con las siguientes columnas:

- To Do: tareas planificadas para el hito actual que aún no se han comenzado.
- In Progress: tareas que están siendo desarrolladas en ese momento.
- Ready for Testing: tareas terminadas pendientes de revisión por otro miembro del equipo.
- Test: tareas en proceso de verificación y validación.
- Done: tareas completamente finalizadas y validadas.

Las tareas se asignan al inicio de cada hito durante la reunión de planificación, y cada miembro es responsable de mover sus tarjetas según avanza. Para evitar sobrecarga establecemos un límite de trabajo en curso (WIP) de 2 tareas activas por persona en la columna In Progress.

Control de versiones con GitHub y SourceTree:

Para el control de versiones del código utilizamos GitHub como repositorio remoto donde se almacena y centraliza todo el código del proyecto, y SourceTree como cliente visual que nos permite gestionar ramas, commits y fusiones de forma gráfica sin usar la línea de comandos directamente. La combinación de ambas herramientas nos facilita trabajar en equipo sin pisarnos el trabajo unos a otros, manteniendo un historial claro de todos los cambios realizados. El control de versiones fue una responsabilidad rotativa entre todos los miembros del equipo, de forma que todos participaron en el mantenimiento del repositorio.

Organización del equipo:

El trabajo se divide al inicio de cada hito según las áreas de responsabilidad de cada miembro, tal y como se detalla en el punto 5.3. Cada persona es responsable de completar sus tareas en el plazo del hito, actualizar el tablero de Trello y subir sus cambios al repositorio.

La coordinación se realiza mediante Weekly Scrum semanales en las que cada miembro informa del estado de sus tareas, comparte posibles bloqueos y acuerda los próximos pasos. El Scrum Master se encarga de convocar estas reuniones, hacer seguimiento del estado general del tablero y alertar al equipo si alguna tarea corre riesgo de no completarse a tiempo.

6. Codificación.

6.1. Entorno de programación

El desarrollo de Scotty App se ha llevado a cabo en un entorno homogéneo: todos los miembros del equipo han trabajado sobre Windows como sistema operativo, lo que ha facilitado la compatibilidad de los archivos y ha evitado problemas derivados de diferencias entre plataformas.

Como entorno de desarrollo integrado (IDE) se ha utilizado IntelliJ IDEA, de JetBrains. Esta elección se justifica por varias razones: es una herramienta profesional ampliamente utilizada en el sector, ofrece soporte nativo para Java lenguaje empleado en el backend del proyecto, e incluye funcionalidades avanzadas como autocompletado inteligente, detección de errores en tiempo real y herramientas de refactorización que agilizan el desarrollo. Además, IntelliJ IDEA permite trabajar con proyectos web, lo que lo convierte en un entorno unificado para las distintas capas del proyecto.

Para el desarrollo de la parte frontend (HTML, CSS y JavaScript) los archivos se han editado directamente desde el mismo IDE y probado abriendo el navegador en local, sin necesidad de un servidor de desarrollo intermedio. Una vez validado el funcionamiento en local, el código se sube al repositorio de GitHub mediante SourceTree y se despliega en el hosting CDMON, que actúa como servidor de producción accesible desde cualquier navegador.

El control de versiones se gestiona con GitHub como repositorio remoto y SourceTree como cliente visual, tal y como se detalla en el apartado 5.4 de este documento.

6.2. Lenguajes y herramientas

En el desarrollo del proyecto se han utilizado diversas tecnologías tanto para el frontend como para el backend, así como para la gestión de datos.

Lenguajes de programación:

- JavaScript: Utilizado en el frontend para dotar de interactividad a la aplicación, permitiendo la manipulación dinámica del DOM y la mejora de la experiencia de usuario.
- Java: Empleado en el backend para la lógica de negocio y la conexión con la base de datos. Se ha elegido por su robustez, amplia documentación y uso extendido en aplicaciones empresariales.
- SQL: Utilizado para la gestión y consulta de la base de datos, permitiendo almacenar y recuperar la información de forma estructurada.

Tecnologías Frontend:

- HTML5: Usado para estructurar el contenido de la aplicación web. Es el estándar base para el desarrollo web.
- CSS3: Utilizado para el diseño y la presentación visual, permitiendo crear una interfaz atractiva y responsive.
- JavaScript (normal): Se ha optado por JavaScript sin frameworks para mantener el proyecto sencillo y ligero, facilitando su comprensión y mantenimiento.

Tecnologías Backend:

- Java (JDK25): Se ha utilizado para desarrollar la lógica del servidor y gestionar la comunicación con la base de datos mediante JDBC. Se eligió por su estabilidad, rendimiento y facilidad de integración con sistemas de bases de datos.

Base de datos:

- Microsoft SQL Server: Sistema gestor de bases de datos relacional utilizado para almacenar la información del proyecto. Se ha elegido por su fiabilidad, potencia y buena integración con herramientas como SQL Server Management Studio.

Herramientas de desarrollo:

- IntelliJ IDEA: Entorno de desarrollo utilizado para programar el backend en Java, debido a sus potentes herramientas de depuración y productividad.
- SQL Server Management Studio (SSMS): Herramienta empleada para la administración y gestión de la base de datos, facilitando la ejecución de consultas y el diseño de tablas.

- Navegador web (Chrome, Firefox, etc.): Utilizado para probar y visualizar el frontend de la aplicación.

Librerías y frameworks: Actualmente, el proyecto no utiliza frameworks ni librerías externas, con el objetivo de mantener una estructura sencilla y centrarse en el uso de tecnologías base. Esto facilita el aprendizaje y la comprensión del funcionamiento interno de la aplicación. Aunque en un futuro por temas de funcionalidad y desarrollo más ágil, implementaremos librerías que nos ayuden a implementar nuevas funcionalidades en la aplicación.

6.3. Aspectos relevantes de la implementación

En esta sección se describen las decisiones técnicas más relevantes tomadas durante el desarrollo del MVP de Scotty App, explicando cómo se han implementado las funcionalidades principales y qué problemas se han resuelto en el proceso.

Estructura del proyecto:

El proyecto frontend está organizado en seis archivos HTML independientes, cada uno correspondiente a una pantalla del flujo de usuario: index.html (pantalla de inicio), signup.html (registro), login.html (inicio de sesión), selector.html (elección de lenguaje), app.html (panel de juego) y la landing page principal con su propio app.js y styles.css. Esta separación por páginas permite una navegación clara y un mantenimiento sencillo del código.

Registro y persistencia de datos en local:

Dado que el MVP no dispone aún de backend conectado, los datos del usuario se almacenan temporalmente en el localStorage del navegador. Cuando el usuario completa el formulario de registro, los datos se recogen y se guardan como un objeto JSON:

```
document.getElementById('regForm').addEventListener('submit', (e) => {
  e.preventDefault();
  const usuario = {
    nombre: document.getElementById('nombre').value,
    email: document.getElementById('email').value,
    tel: document.getElementById('tel').value,
    pass: document.getElementById('pass').value,
    lenguaje: sessionStorage.getItem('lenguajeElegido') || 'General'
  };
  localStorage.setItem('usuarioRegistrado', JSON.stringify(usuario));
  window.location.href = "login.html";
});
```

Esta solución permite demostrar el flujo completo de registro sin necesidad de servidor. El lenguaje elegido en la pantalla anterior se recupera del sessionStorage, lo que muestra la comunicación entre páginas mediante almacenamiento web del navegador.

Sistema de login con verificación local:

La pantalla de login recupera el objeto guardado en localStorage y compara las credenciales introducidas con las almacenadas. Si coinciden, redirige al panel de juego; si no, muestra un mensaje de error:

```
function login() {  
  const user = JSON.parse(localStorage.getItem('usuarioRegistrado'));  
  const email = document.getElementById('lEmail').value;  
  const pass = document.getElementById('lPass').value;  
  if (user && user.email === email && user.pass === pass) {  
    window.location.href = "app.html";  
  } else {  
    document.getElementById('msg').innerText = "Usuario o contraseña incorrectos";  
  }  
}
```

Es importante señalar que este sistema de autenticación es una solución funcional para el MVP académico. En una versión real, las contraseñas nunca se almacenarían en texto plano en el navegador, sino que se gestionarían en el servidor con hash y cifrado.

Validación del formulario de la landing page:

La landing page incluye un formulario de registro con validación en el cliente mediante JavaScript puro, sin librerías externas. La validación se ejecuta al enviar el formulario e impide el envío si algún campo no cumple los requisitos:

```
// Validar nombre (mínimo 3 caracteres)  
if (nombre.value.trim().length < 3) {  
  errorNombre.classList.remove("hidden");  
  nombre.classList.add("error");  
  hayErrores = true;  
}  
  
// Validar email con expresión regular  
var emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;  
if (!emailRegex.test(email.value.trim())) {  
  errorEmail.classList.remove("hidden");  
  email.classList.add("error");  
  hayErrores = true;  
}  
  
// Validar contraseña (mínimo 8 caracteres)  
if (password.value.length < 8) {  
  errorPassword.classList.remove("hidden");  
  password.classList.add("error");  
  hayErrores = true;  
}
```

Cada campo recibe retroalimentación visual inmediata mediante clases CSS: borde rojo y mensaje de error si el valor no es válido, borde verde si es correcto. Si todos los campos superan la validación, se muestra un mensaje de éxito y el formulario se resetea.

Flujo de navegación y control de sesión:

La pantalla app.html comprueba al cargarse si existe un usuario registrado en localStorage. Si no lo hay, redirige automáticamente a la página de inicio, evitando el acceso sin autenticación:

```
const u = JSON.parse(localStorage.getItem('usuarioRegistrado'));  
if (!u) window.location.href = "index.html";
```

Una vez autenticado, el panel muestra los datos del perfil del jugador (nombre, lenguaje elegido y contacto) recuperados del objeto almacenado, y presenta la primera misión interactiva del juego.

Problemas encontrados y soluciones:

El principal reto durante esta fase fue la ausencia de backend operativo en el plazo del Hito 4. La solución adoptada fue utilizar localStorage y sessionStorage para simular la persistencia de datos, lo que permitió completar el flujo de usuario completo (registro → login → selector → juego) de forma funcional y demostrable. Esta solución tiene limitaciones evidentes (los datos se pierden si el usuario borra el almacenamiento del navegador y no son compartidos entre dispositivos), pero cumple el objetivo de validar la experiencia de usuario del MVP.

7. Plan de Negocio

7.1. Público Objetivo

- El público objetivo principal de Scotty App son estudiantes de ESO, Bachillerato y Formación Profesional, especialmente aquellos que están empezando a aprender programación o que tienen dificultades para seguir el ritmo en clase. También está orientado a alumnos que pierden interés rápidamente cuando el contenido es demasiado teórico.
- Por otro lado, la aplicación también va dirigida a centros educativos que incluyan asignaturas de programación en su plan de estudios, ya que puede servir como apoyo al contenido que se imparte en clase. No sustituye al profesor, pero sí puede complementar bastante bien.
- Además, los profesores son otro perfil importante, ya que pueden utilizar la herramienta como recurso adicional para hacer las clases más dinámicas o para reforzar ciertos conceptos que a veces cuesta explicar solo con teoría.

7.2. Utilidad Social o Empresarial — (José Luis)

- En cuanto a su utilidad social, Scotty App busca principalmente mejorar la motivación del alumnado. Muchas veces el problema no es la dificultad en sí, sino cómo se presenta el contenido, y una aplicación más interactiva puede ayudar bastante en ese sentido. También puede contribuir a reducir el abandono en asignaturas relacionadas con la programación, que suelen ser vistas como complicadas.
- Respecto a la utilidad empresarial, el proyecto podría ofrecerse como un servicio a centros educativos, ya sea como herramienta de apoyo o integrada dentro de sus recursos digitales. También existe la posibilidad de desarrollar una versión premium con más funcionalidades, como más niveles, seguimiento más detallado o contenido adicional.
- Por último, es un proyecto que podría escalarse sin demasiados problemas a otros centros o incluso a nivel de comunidades, aunque eso ya dependería de cómo evolucione en el futuro.

7.3. Posibles Costes y Beneficios

Costes:

- Desarrollo inicial (tiempo del equipo).
- Servidor y alojamiento web.

Beneficios potenciales:

- Suscripción de centros educativos.
- Posible versión gratuita con publicidad o versión premium.

8. Manual de Usuario

8.1. Descripción general de la aplicación

Scotty App es una plataforma web educativa y gamificada diseñada para que estudiantes de ESO, Bachillerato y Formación Profesional aprendan lenguajes de programación (HTML, CSS, JavaScript, Java y SQL) de forma práctica.

La aplicación transforma el aprendizaje teórico en una experiencia de juego mediante retos interactivos, un sistema de puntos de experiencia (XP), niveles progresivos y recompensas digitales.

8.2. Acceso al sistema

Para acceder a Scotty App, el usuario debe entrar a través de un navegador web.

- Modo Invitado: Cualquier persona puede explorar el catálogo de juegos y probar retos de forma temporal sin registrarse.
- Nota importante: En el modo invitado, el progreso y la puntuación no se guardan al cerrar el navegador.

8.3. Registro e inicio de sesión

Para disfrutar de la experiencia completa y guardar el progreso, es necesario crear una cuenta:

- Registro (Sign Up): En el formulario de registro, el usuario debe facilitar su nombre, correo electrónico y una contraseña de al menos 8 caracteres.
- Inicio de sesión (Login): Una vez registrado, el usuario accede con su correo y contraseña. El sistema verificará las credenciales y, si son correctas, le redirigirá a su panel personal.
- Gestión de Perfil: El alumno puede editar sus datos, como el nombre de usuario y su avatar, desde la configuración de su perfil.

8.4. Navegación por la aplicación

La interfaz de Scotty App es responsive, adaptándose a ordenadores, tablets y móviles. Los elementos principales de navegación incluyen:

- Mapa de Niveles: Un recorrido visual donde se seleccionan los lenguajes y los retos disponibles.
- Selector de Lenguaje: Pantalla para elegir entre las diferentes tecnologías (JS, Java, SQL, etc.) antes de comenzar una misión.
- Panel de Progreso: Un espacio donde el usuario visualiza su nivel actual, la XP acumulada, sus rachas y las insignias (badges) obtenidas.
- Tienda y Ranking: Secciones para canjear "tokens" por recompensas y comparar la puntuación con otros alumnos en el ranking global.

8.5. Uso de las funcionalidades principales

El núcleo de la aplicación son los Retos Interactivos:

- Completar Retos: El usuario se enfrenta a ejercicios prácticos (de código, bloques o selección). Al resolverlos correctamente, recibe feedback visual inmediato (bordes verdes para aciertos) y acumula XP y tokens.
- Progreso de Niveles: A medida que se gana XP, el alumno sube de nivel, desbloqueando nuevos retos más complejos en el mapa.
- Funciones para Profesores: Los usuarios con perfil de profesor pueden, además, crear retos personalizados para sus clases, organizar a los alumnos en equipos y seguir su evolución mediante un panel de seguimiento específico.
- Canje de Recompensas: Los tokens acumulados pueden ser canjeados en la tienda virtual por avatares u otros elementos estéticos que se reflejarán en el perfil del usuario.

9. Requisitos e Instalación

9.1. Requisitos del sistema

Para ejecutar este proyecto es necesario contar con el siguiente entorno:

Hardware:

- Procesador: Intel i5 o equivalente
- Memoria RAM: mínimo 8 GB recomendados
- Espacio en disco: al menos 1GB disponible

Software:

- Sistema operativo:
 - o Windows 10 o superior (recomendado)
 - o Compatible con macOS y Linux
- Navegador web moderno:
 - o Google Chrome (recomendado)
 - o Mozilla Firefox
 - o Microsoft Edge

- Backend:
 - Java Development Kit (JDK 25)
 - IntelliJ IDEA (u otro IDE compatible)
 - Base de datos:Sistema de gestor SQL: Microsoft SQL Server

9.2. Instalación y ejecución

Sigue estos pasos para ejecutar el proyecto en un entorno local:

1. Clonar o descargar el proyecto

`git clone https://github.com/Scotty-App` (en proceso)

`cd proyecto`

(O descargar y descomprimir manualmente si no se usa Git)

2. Configuración de la base de datos

1. Crear una base de datos en el gestor elegido (MySQL, PostgreSQL, etc.)
2. Importar el script SQL del proyecto (si existe)
3. Configurar la conexión en el backend (Java), normalmente en:
 - a. archivo de configuración
 - b. clase de conexión

Ejemplo de parámetros típicos:

- URL: `jdbc:mysql://localhost:3306/nombre_bd`
- Usuario: `root`
- Contraseña: `*****`

3. Ejecución del frontend

El frontend es estático, por lo que puede ejecutarse directamente:

- Abrir el archivo `index.html` en un navegador web

O alternativamente:

- Hacer doble clic sobre el archivo
- O abrirlo desde el navegador (Chrome, Firefox, etc.)

4. Ejecución del backend

1. Abrir el proyecto en IntelliJ IDEA
2. Esperar a que se carguen las dependencias
3. Configurar la conexión a la base de datos en el código
4. Ejecutar la clase principal del proyecto (main)

5. Funcionamiento

- El frontend se ejecuta en local desde el navegador
- El backend se ejecuta desde el entorno de desarrollo (IDE)
- La comunicación con la base de datos se realiza mediante Java (JDBC)

Notas adicionales

- Actualmente el proyecto está preparado para ejecución en entorno local
- No requiere despliegue en servidor ni configuración de puertos
- No utiliza contenedores Docker ni sistema de testing automatizado

10. Conclusiones

10.1. Conclusiones sobre el trabajo realizado

El desarrollo de Scotty App ha permitido aplicar de forma práctica los conocimientos adquiridos durante el curso, especialmente en la planificación, organización y análisis del proyecto. El uso de herramientas como Trello y GitHub ha sido clave para coordinar el trabajo en equipo y mantener un seguimiento claro de las tareas. Como resultado, se ha definido una propuesta coherente y enfocada a resolver un problema real: la falta de motivación en el aprendizaje de la programación. En general, el proyecto demuestra ser viable y con potencial de desarrollo futuro.

10.2. Posibles ampliaciones y mejoras

El MVP de Scotty App cubre las funcionalidades esenciales definidas en el alcance del proyecto, pero el sistema está diseñado para crecer de forma ordenada. A continuación se detallan tres ampliaciones que consideramos realistas y coherentes con la naturaleza del proyecto.

Mentor con inteligencia artificial: La incorporación de un asistente de IA integrado en el editor de retos permitiría ofrecer al alumno pistas personalizadas basadas en su código, sin darle directamente la solución. El sistema analizaría los errores más frecuentes del usuario y adaptaría las sugerencias a su nivel de progreso, convirtiendo la plataforma en una experiencia de aprendizaje verdaderamente personalizada.

Mejoras de seguridad: En una versión posterior se reforzaría el sistema de autenticación añadiendo doble factor (2FA) y una gestión de sesiones más robusta mediante tokens JWT de corta duración con refresh tokens automáticos. Estas mejoras son especialmente relevantes si la plataforma llega a gestionar datos de menores de edad en entornos educativos reales.

Ejecución real de código en sandbox: Actualmente la validación de respuestas se basa en comparación estática. Una mejora técnica prioritaria sería ejecutar el código del usuario en un entorno aislado, lo que haría los retos mucho más robustos y permitiría ampliar el catálogo a lenguajes que requieren compilación o ejecución real, como Java o Python.

11. Anexos: Diseño Preliminar

11.1. Descripción Paso a Paso del Caso de Uso Principal

El caso de uso principal de Scotty App es "Jugar un reto con guardado de progreso", ya que representa la interacción central entre el alumno y la plataforma.

Actor principal: Alumno (usuario registrado y autenticado).

Precondiciones: El alumno ha iniciado sesión en la plataforma.

Postcondiciones: El progreso del alumno se actualiza en la base de datos (XP ganada, nivel completado, tokens acumulados).

Flujo normal:

1. El alumno accede al dashboard principal donde se muestra el mapa de niveles y juegos disponibles.
2. El alumno selecciona un juego del catálogo (por ejemplo, "Introducción a HTML").
3. El sistema muestra los niveles del juego, indicando cuáles están desbloqueados, en progreso o completados.
4. El alumno selecciona un nivel desbloqueado.
5. El sistema carga el primer reto del nivel y muestra la interfaz de juego: enunciado a la izquierda, área de resolución a la derecha y la mascota Scotty dando feedback visual.
6. El alumno lee el enunciado y selecciona o escribe su respuesta.
7. El alumno pulsa el botón de enviar respuesta.
8. El sistema evalúa la respuesta mediante el método `validarRespuesta()`.
9. Si la respuesta es correcta, el sistema muestra feedback positivo, acumula XP y avanza al siguiente reto.
10. Al completar todos los retos del nivel, el sistema marca el nivel como COMPLETADO, otorga los tokens de recompensa y desbloquea el siguiente nivel.

Flujos alternativos:

Paso 9a — Respuesta incorrecta: El sistema muestra la respuesta correcta resaltada en verde, muestra un mensaje de feedback y permite continuar con el siguiente reto sin penalización de XP.

Paso 4a — Nivel bloqueado: Si el alumno intenta acceder a un nivel no desbloqueado, el sistema muestra un mensaje indicando que debe completar los niveles anteriores.

Paso 1a — Sesión expirada: Si la sesión ha expirado, el sistema redirige a la pantalla de login y, tras autenticarse, le devuelve al punto donde se encontraba.

11.2. Mockups de 2–3 Pantallas Clave

En esta etapa del trabajo, el equipo ha utilizado Figma para hacer un dibujo detallado de cómo será la pantalla principal. Hacer este "plano" es fundamental antes de empezar a escribir código, ya que ayuda a tener claro qué hay que construir y evita perder el tiempo corrigiendo fallos sobre la marcha.

Figura 7 - Mockup de pantallas clave



11.3. Prevención de Riesgos Laborales (PRL)

El desarrollo del proyecto se realiza de forma telemática, lo que implica que cada miembro del equipo trabaja desde su domicilio particular. Esta modalidad no exime de los riesgos asociados al uso de Pantallas de Visualización de Datos (PVD) y requiere una gestión específica de los materiales y el entorno.

11.3.1. Identificación de riesgos asociados al desarrollo (Teletrabajo).

- Siguiendo los bloques de salud laboral del manual de IPE I (pág. 22), se han identificado los siguientes riesgos adaptados a nuestro entorno doméstico:
- Riesgos derivados del espacio de trabajo: Al trabajar en domicilios, existe el riesgo de contar con una iluminación inadecuada o reflejos en la pantalla que aumentan la fatiga visual.
- Riesgos ergonómicos: El uso de mobiliario no profesional (mesas bajas o sillas no regulables) incrementa la probabilidad de sufrir trastornos musculoesqueléticos en la zona lumbar y cervical.
- Riesgos psicosociales (Aislamiento y Desconexión): El trabajo a distancia puede generar estrés por el cumplimiento de hitos y la dificultad para separar la vida personal de la académica, derivando en fatiga mental.

11.3.2. Medidas preventivas aplicables al trabajo con equipos informáticos

Para mitigar estos riesgos, el equipo aplica los principios de la acción preventiva (pág. 65):

- **Higiene postural:** Se sigue la guía de "La postura de trabajo frente al ordenador" (pág. 40), manteniendo la pantalla a la altura de los ojos y los antebrazos apoyados.
- **Equipos de Protección Individual (EPI):** Aunque en el desarrollo de software no se utilizan EPI tradicionales como cascos, el equipo considera como tales las muñequeras ergonómicas para prevenir el síndrome del túnel carpiano y, si fuera necesario por la fatiga ocular, filtros de luz azul en las PVD.
- **Pausas y Gimnasia Laboral:** Se establecen descansos de 5-10 minutos por cada hora de trabajo para realizar estiramientos y relajar la acomodación visual.

11.3. 3. Facilitación de material y responsabilidades

Atendiendo a los derechos y deberes en materia de PRL recogidos en la Ley 31/1995 y el manual de IPE I (pág. 51):

- Provisión de materiales: La normativa establece que es la organización (en un entorno laboral, la empresa) quien tiene la obligación de facilitar y costear los medios necesarios para el desempeño del trabajo seguro, incluyendo el equipo informático y los elementos ergonómicos.
- Coste de las medidas: Según el principio de gratuidad de la prevención para el trabajador, el coste de las medidas de seguridad y salud no debe recaer sobre los miembros del equipo.
- Información y formación: Cada miembro del grupo es responsable de informarse sobre el uso correcto de los equipos y reportar cualquier anomalía técnica que pueda suponer un riesgo para su salud.

12. Seguimiento del Proyecto (enlaces externos de gestión)

Además del desarrollo documental, el equipo ha llevado a cabo un seguimiento real del proyecto mediante herramientas de gestión de tareas y control de versiones. Este seguimiento permite demostrar la implicación individual de cada miembro, la organización del equipo y la evolución real del proyecto a lo largo de los hitos.

12.1. URL del Tablero de Trello (Gestión de tareas)

Para la gestión y organización del trabajo en equipo, se utiliza un tablero en Trello. Esta herramienta permite aplicar de forma sencilla el método Kanban, visto en el módulo de Entornos de Desarrollo, para repartir las tareas y saber en todo momento qué está haciendo cada miembro del grupo.

URL del Tablero: <https://trello.com/b/sttwpxvy/scotty>

Figura 8 - Captura del tablero Trello



12.2. URL del Repositorio de GitHub

La gestión técnica y el control de versiones del proyecto se realizan mediante Git, y utilizaremos GitHub como servidor remoto para almacenar y compartir el progreso de la aplicación.

Organización y Repositorio Principal: Para centralizar el trabajo de los cinco integrantes del equipo (José Luis, José Ramón, Iván y Manuela), se ha habilitado una organización profesional que permite administrar los repositorios y permisos de forma conjunta:

- URL de la Organización: <https://github.com/Scotty-App>
- Nombre del Repositorio: La documentación técnica residen actualmente en el repositorio Documentación.

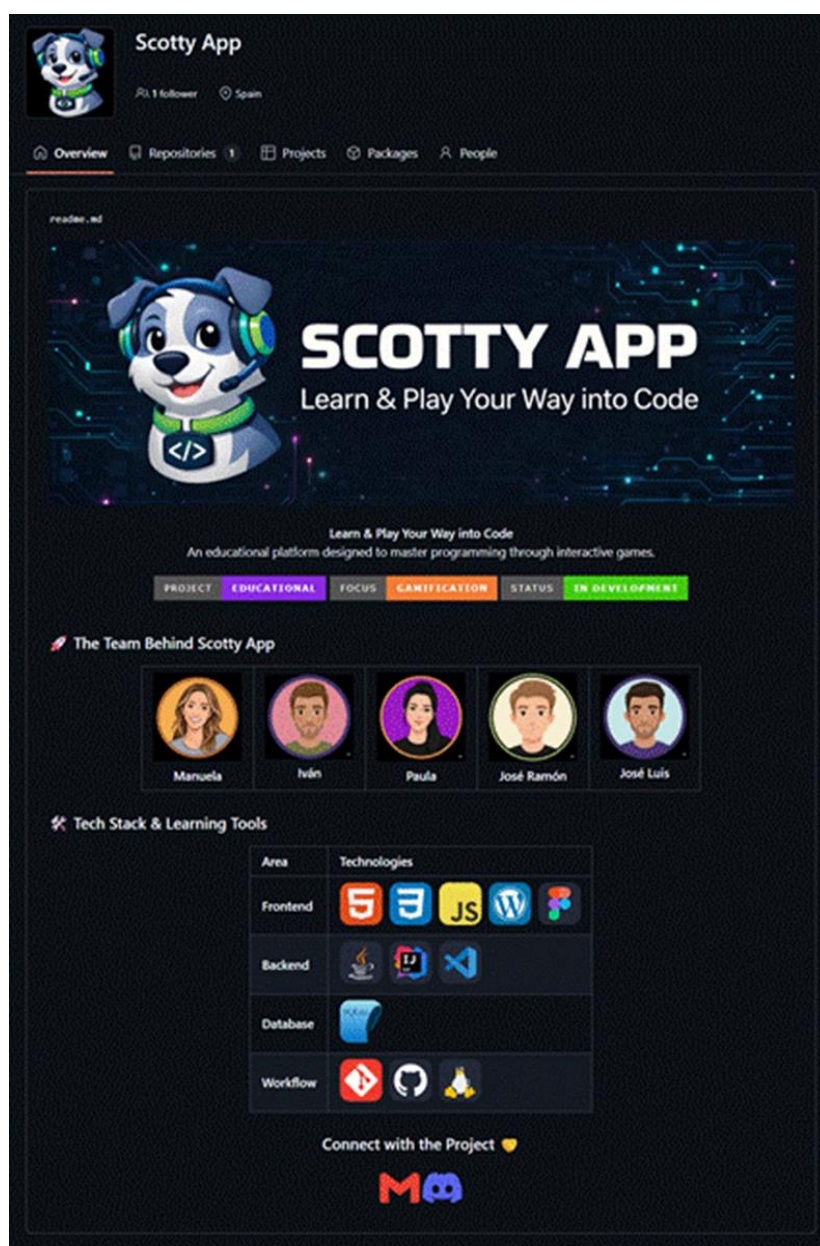
Fichero README.md y Stack Tecnológico: En la raíz del repositorio se ha incluido un archivo README.md, el cual actúa como presentación del proyecto y permite visualizar la información básica del sistema directamente en GitHub. Según lo definido en dicho archivo, el proyecto presenta las siguientes características:

- Nombre del Grupo: Scotty App.
- Descripción: Plataforma educativa diseñada para el aprendizaje de programación mediante juegos interactivos.

- Equipo: Se muestran los miembros del equipo.
- Tech Stack: Se han seleccionado tecnologías estándar de la industria, como HTML5, CSS3, JavaScript para el frontend, Java para el backend y SQL Server para la persistencia de datos.

Acceso y Evaluación: Para garantizar que el desarrollo sea accesible para el profesorado y se pueda comprobar el avance semanal, se ha invitado formalmente al docente a través de su correo institucional (sdianezjuan@edu.gva.es). De este modo, el historial de commits sirve como evidencia de la participación individual de cada miembro del grupo.

Figura 9 - Captura del repositorio GitHub



12.2.1. Gestión de Ramas y Commits.

El equipo utiliza una estructura sencilla en GitHub. La rama principal es main, donde se mantiene la versión estable del proyecto. Normalmente se trabaja directamente sobre esta rama para la documentación, y solo se crean otras ramas cuando es necesario trabajar en algo concreto.

Los commits se hacen con mensajes claros indicando qué se ha cambiado, evitando mensajes genéricos. Además, se intenta trabajar de forma progresiva y no dejar todo para el final.

Se utiliza SourceTree para gestionar los cambios de forma más visual y sencilla.

12.3. Control de Versiones y README.md

El repositorio del proyecto se gestiona a través de GitHub bajo la organización Scotty-App. El fichero README.md, situado en la raíz del repositorio, actúa como carta de presentación del proyecto y contiene la información esencial: nombre del grupo, descripción del proyecto, integrantes del equipo y stack tecnológico.

El README.md del repositorio de base de datos incluye además documentación específica sobre la estructura de la base de datos: tecnología utilizada (Microsoft SQL Server con T-SQL), tipos de usuario contemplados (admin, normal, invitado), descripción de cada una de las 9 tablas, el modelo relacional en formato texto y las relaciones principales entre tablas con su cardinalidad.

El equipo sigue estas prácticas de control de versiones: commits con mensajes descriptivos que indican claramente qué cambio se ha realizado, historial progresivo distribuido en el tiempo evitando concentrar el trabajo en un único día, y uso de SourceTree como cliente visual para facilitar la gestión de ramas y fusiones.

1. Bibliografía y webgrafía

Entornos de Desarrollo

Diáñez Juan, S. (Curso 2025/2026). *UP 1 — Desarrollo del software. Fases del ciclo de vida: análisis, diseño, codificación, pruebas, documentación, explotación y mantenimiento*. IES Mutxamel.

Diáñez Juan, S. (Curso 2025/2026). *UP 4 — Optimización y documentación. Control de versiones y repositorios*. IES Mutxamel.

Diáñez Juan, S. (Curso 2025/2026). *UP 5 — Elaboración de diagramas de clases. UML, relaciones, herencia, visibilidad*. IES Mutxamel.

Diáñez Juan, S. (Curso 2025/2026). *UP 6 — Elaboración de diagramas de comportamiento. Diagramas de casos de uso, actores, relaciones «include» y «extend»*. IES Mutxamel.

Schwaber, K., y Sutherland, J. (2020). *La Guía de Scrum. Las reglas del juego* (versión noviembre 2020). Scrum.org. Recuperado de <https://scrumguides.org>

Figma Inc. (2026). *Documentación oficial. Herramienta de diseño colaborativo utilizada para los mockups y el diagrama de arquitectura*. Recuperado de <https://www.figma.com>

Sistemas Informáticos

Vera Lloret, L. J. (Curso 2025/2026). *Sistemas Informáticos (SIN). Arquitectura de sistemas web: componentes, capas y flujo de datos*. IES Mutxamel.

Bases de Datos

Fraila Antón, M. V. (Curso 2025/2026). *Módulo Bases de Datos — RA2: ER Model y Design an ER Diagram. RA2: Relational Model. RA2: Data Definition Language. Script SQL de creación de tablas*. IES Mutxamel.

Programación

Benavente Domenech, P. (Curso 2025/2026). *Módulo Programación. Programación orientada a objetos en Java, acceso a datos y programación gráfica*. IES Mutxamel.

JUnit Team. (2026). *Documentación oficial de JUnit 5. Framework de pruebas unitarias para Java*. Recuperado de <https://junit.org/junit5>

Oracle Corporation. (2026). *Documentación oficial de Java. JavaDoc y generación de documentación de código*. Recuperado de <https://docs.oracle.com/en/java>

Atlassian. (2026). *SourceTree. Cliente visual para gestión de repositorios Git*. Recuperado de <https://www.sourcetreeapp.com>

GitHub Inc. (2026). *Documentación oficial de GitHub. Control de versiones, ramas y commits*. Recuperado de <https://docs.github.com>

Lenguaje de Marcas

López Coloma, F. (Curso 2025/2026). *Módulo Lenguaje de Marcas — RA2: HTML y CSS. RA3: JavaScript. RA6: BASEX. RA7: ERP, CRM, SaaS. IES Mutxamel.*

W3Schools. (2026). *Tutoriales de HTML5, CSS3 y JavaScript. Referencia técnica utilizada en el desarrollo de la landing page.* Recuperado de <https://www.w3schools.com>

MDN Web Docs — Mozilla. (2026). *Documentación web de referencia: HTML, CSS y JavaScript.* Recuperado de <https://developer.mozilla.org>

Itinerario Personal para la Empleabilidad (IPE)

Caldas Blanco, M. E., Hidalgo Ortega, M. L., y Laraño Díaz, J. E. (2024). *Itinerario Personal para la Empleabilidad I* (ISBN: 978-84-1134-936-9). Editorial Editex, S. A.

Ley 31/1995, de 8 de noviembre, de Prevención de Riesgos Laborales. *Boletín Oficial del Estado*, núm. 269, de 10 de noviembre de 1995.

Real Decreto 488/1997, de 14 de abril, sobre disposiciones mínimas de seguridad y salud relativas al trabajo con equipos que incluyen pantallas de visualización. *Boletín Oficial del Estado*, núm. 97, de 23 de abril de 1997.

Benchmarking, inspiración y referencias de diseño

Kahoot! (2026). *Plataforma de aprendizaje gamificado. Analizada como referencia de mecánicas de motivación y engagement.* Recuperado de <https://kahoot.com>

CodeCombat. (2026). *Plataforma de aprendizaje de programación mediante videojuegos. Analizada como referencia de estructura de retos y progresión.* Recuperado de <https://codecombat.com>

SoloLearn. (2026). *Plataforma de aprendizaje de lenguajes de programación. Analizada como referencia de catálogo de contenidos educativos.* Recuperado de <https://www.sololearn.com>

Codecademy. (2026). *Plataforma de aprendizaje interactivo de programación. Referencia para el diseño de niveles y sistema de progresión de Scotty App.* Recuperado de <https://www.codecademy.com>

OpenAI. (2026). *ChatGPT. Modelo de inteligencia artificial utilizado como apoyo en la redacción, estructuración de contenido y generación de imágenes del proyecto.* Recuperado de <https://chat.openai.com>

Anthropic. (2026). *Claude. Modelo de inteligencia artificial utilizado como apoyo en el análisis, revisión de coherencia documental y generación del diagrama de arquitectura.* Recuperado de <https://claude.ai>

Atlassian. (2026). *Trello. Herramienta de gestión de tareas utilizada para el seguimiento del proyecto mediante tablero Kanban.* Recuperado de <https://trello.com>